

Desenvolvimento Iterativo, Colaboração com IA e Validação Automatizada

CampusFlow

Entrega 1 — Ambiente, Especificação Técnica e Test Harness

Integrante: Felipe Amorim Monteiro

RA: 22452139

Repositório público:

<https://github.com/MorimFr/entrega-inicial>

Data: 1 de setembro de 2026

Entrega validada

1. Visão geral

O CampusFlow é uma API para reserva de salas de estudo compartilhadas. O sistema elimina conflitos de horário, respeita a capacidade das salas, limita duração e quantidade diária de reservas e mantém um contrato HTTP verificável.

Escopo da primeira iteração

- listar salas e respectivas capacidades;
- criar, consultar e cancelar reservas;
- impedir sobreposição de intervalos;
- limitar reservas a duas horas e duas reservas ativas por usuário/dia;
- consultar disponibilidade por sala e intervalo.

Tecnologias

Área	Tecnologia	Finalidade
Aplicação	Python 3.12, FastAPI, Pydantic	API, contratos e validação de entrada.
Qualidade	Pytest, pytest-cov, Ruff	Testes, cobertura e análise estática.
Ambiente	Docker e Docker Compose	Execução reproduzível.
Automação	GitHub Actions	Validação obrigatória em Pull Requests.
IA	Codex e AGENTS.md	Apoio à decomposição, código e testes no fluxo SDD.

Resultado: aplicação funcional, 26 testes automatizados, cobertura de 99,48% no ambiente oficial, CI verde e governança baseada em Issues, branches protegidas, revisões e Pull Requests.

2. Especificação SDD e arquitetura

A especificação em docs/SPECIFICATION.md é a fonte de verdade do comportamento. Ela contém requisitos RF-01 a RF-06, regras RN-01 a RN-09, requisitos não funcionais, contratos JSON/HTTP e uma matriz de rastreabilidade para os testes.

Refinamento orientado por feedback

A revisão dos casos de teste revelou ambiguidades sobre fuso horário, intervalos adjacentes, cancelamento repetido e limite diário. As decisões foram incorporadas à versão 0.2.0 e registradas em docs/SPEC_CHANGELOG.md. A execução Docker também revelou que o atalho pytest não ativava o coletor de cobertura no contêiner; o Compose foi refinado para python -m pytest e validado novamente.

Componentes



Componente	Responsabilidade
api.py	Rotas, injeção e tradução de erros para HTTP.
service.py	Casos de uso e regras de negócio.
domain.py	Entidades e estados sem dependência do framework.
repository.py	Porta de persistência e adaptador em memória.
schemas.py	Contratos públicos de entrada e saída.

As decisões estruturais estão documentadas nos ADRs 001 e 002. Essa separação permite testar as regras sem servidor ou banco e trocar a persistência em uma iteração futura.

3. Ambiente e colaboração com IA

Execução local

```
.\scripts\setup.ps1
.\scripts\test.ps1
uvicorn campusflow.api:app --reload
```

Execução padronizada

```
docker compose up --build api
docker compose --profile test run --rm --build tests
```

O Dockerfile usa Python 3.12 e instala versões fixadas no `pyproject.toml`. O Compose possui serviços independentes para API e testes. A mesma suíte é executada localmente, no contêiner e no GitHub Actions.

Agente Codex

O Codex foi usado como agente auxiliar para decompor o enunciado, estruturar a aplicação e apoiar os testes. O arquivo `AGENTS.md` mantém no repositório as instruções do fluxo SDD, os limites arquiteturais, os comandos de validação e os requisitos de segurança. Essa configuração segue o mecanismo de instruções persistentes documentado oficialmente pelo Codex.

1. Vincular alteração a uma Issue e aos IDs da especificação.
2. Escrever ou ajustar o teste do critério de aceitação.
3. Implementar a menor mudança compatível com o contrato.
4. Executar Ruff, Pytest e cobertura.
5. Submeter a Pull Request e revisão humana.

Prompts e controles de revisão estão registrados em `docs/AI_USAGE.md`. Saída de IA não substitui especificação, teste, CI ou aprovação humana.

4. Test harness e resultados

O harness possui testes unitários do serviço e testes de integração dos contratos HTTP. Cada teste usa um repositório isolado, não depende de rede, relógio real ou ordem de execução, e o pipeline falha se a cobertura ficar abaixo de 90%.

Ambiente	Python	Testes	Cobertura	Estado
Local Windows	3.14.7	26/26	99,39%	Aprovado
Docker Linux	3.12.14	26/26	99,48%	Aprovado
GitHub Actions	3.12.14	26/26	99,48%	Aprovado

Cenários cobertos

- criação, consulta e cancelamento de reserva;
- sala e reserva inexistentes;
- período inválido e ausência de fuso horário;
- fronteira de duas horas e excesso de duração;
- capacidade, sobreposição e intervalos adjacentes;
- limite diário, cancelamento repetido e liberação do horário;
- saúde, catálogo e disponibilidade pela API.

```
Required test coverage of 90% reached. Total coverage: 99.48%
===== 26 passed in 0.52s =====
```

5. Evidência — execução no Docker

```
Problems | Output | Debug Console | Terminal | Ports | Comments | Azure | Query Results

PS C:\Users\Monteiro\Documents\ceub\entrega-inicial> docker compose --profile test run --rm --build tests
#12 exporting manifest sha256:a569a52f942c18b73f28ea962279b9e63e62f13bb708152e5c1480a4b767ec1d done
#12 exporting config sha256:8a976f453d4e08c1726a34e3aa3874354ccf9e53e4c30cb22923ad938f5e14f0 done
#12 exporting attestation manifest sha256:94ab769ec950a4b05124da6a4ab2af395531048eb7286a4491aa98a825df3778 0.0s done
#12 exporting manifest list sha256:f7fc138ca98439cd9017bb0f3c73e54b401b049ef9ffbed8820a9fd6d78ce04 0.0s done
#12 naming to docker.io/library/entrega-inicial-tests:latest
#12 naming to docker.io/library/entrega-inicial-tests:latest done
#12 unpacking to docker.io/library/entrega-inicial-tests:latest 0.0s done
#12 DONE 0.2s

#13 resolving provenance for metadata file
#13 DONE 0.0s
Image entrega-inicial-tests Built
Container entrega-inicial-tests-run-34273fefb2f7 Creating
Container entrega-inicial-tests-run-34273fefb2f7 Created
===== test session starts =====
platform linux -- Python 3.12.14, pytest-8.4.1, pluggy-1.6.0
rootdir: /app
configfile: pyproject.toml
testpaths: tests
plugins: cov-6.2.1, anyio-4.14.2

quest #10
```

Figura 1 — Construção da imagem padronizada e execução do contêiner de testes em Python 3.12.

```
Problems | Output | Debug Console | Terminal | Ports | Comments | Azure | Query Results

PS C:\Users\Monteiro\Documents\ceub\entrega-inicial> docker compose --profile test run --rm --build tests

Name                               Stmts  Miss  Cover   Missing
-----
campusflow/_init__.py              0      0  100%
campusflow/api.py                  46      0  100%
campusflow/domain.py               20      0  100%
campusflow/errors.py               18      0  100%
campusflow/repository.py           29      1   97%    72
campusflow/schemas.py             31      0  100%
campusflow/service.py              47      0  100%
-----
TOTAL                             191      1   99%
Coverage XML written to file coverage.xml
Required test coverage of 90% reached. Total coverage: 99.48%
===== 26 passed in 0.56s =====

PS C:\Users\Monteiro\Documents\ceub\entrega-inicial> 
```

Figura 2 — Resultado final: 26 testes aprovados, cobertura de 99,48% e exit code 0.

6. Evidência — pipeline automatizado

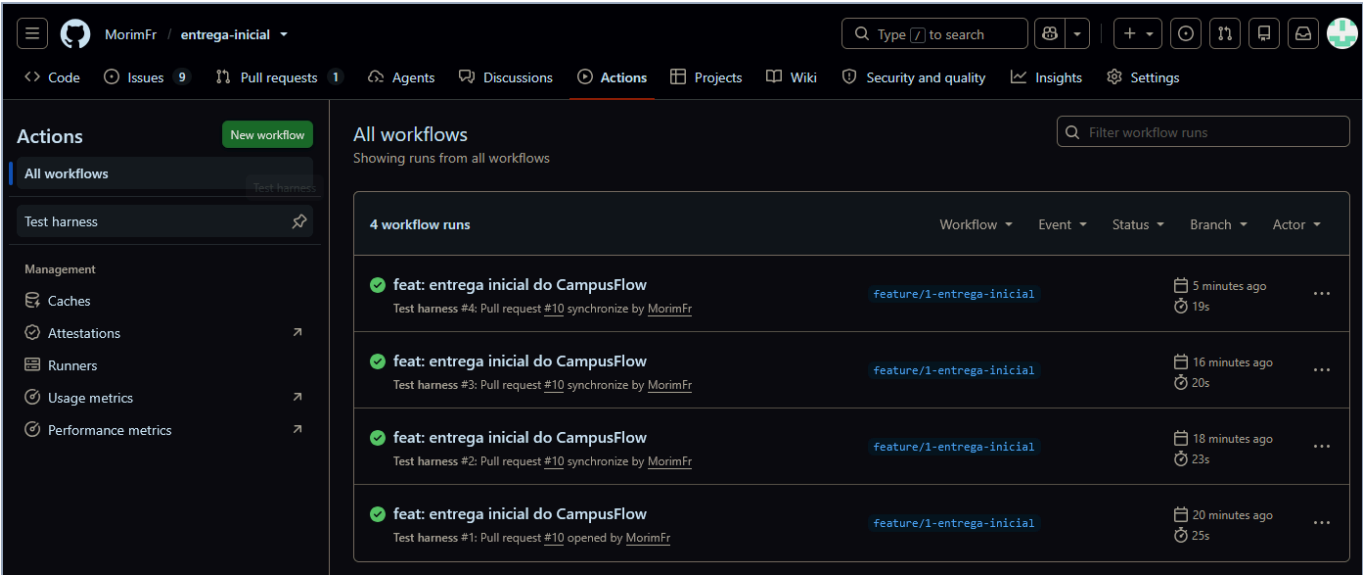


Figura 3 — Workflow Test harness executado automaticamente no PR, com todas as execuções verdes. A execução final está disponível em <https://github.com/MorimFr/entrega-inicial/actions/runs/33578395082> e publicou log e coverage.xml como artefatos.

7. Evidência — planejamento da Sprint

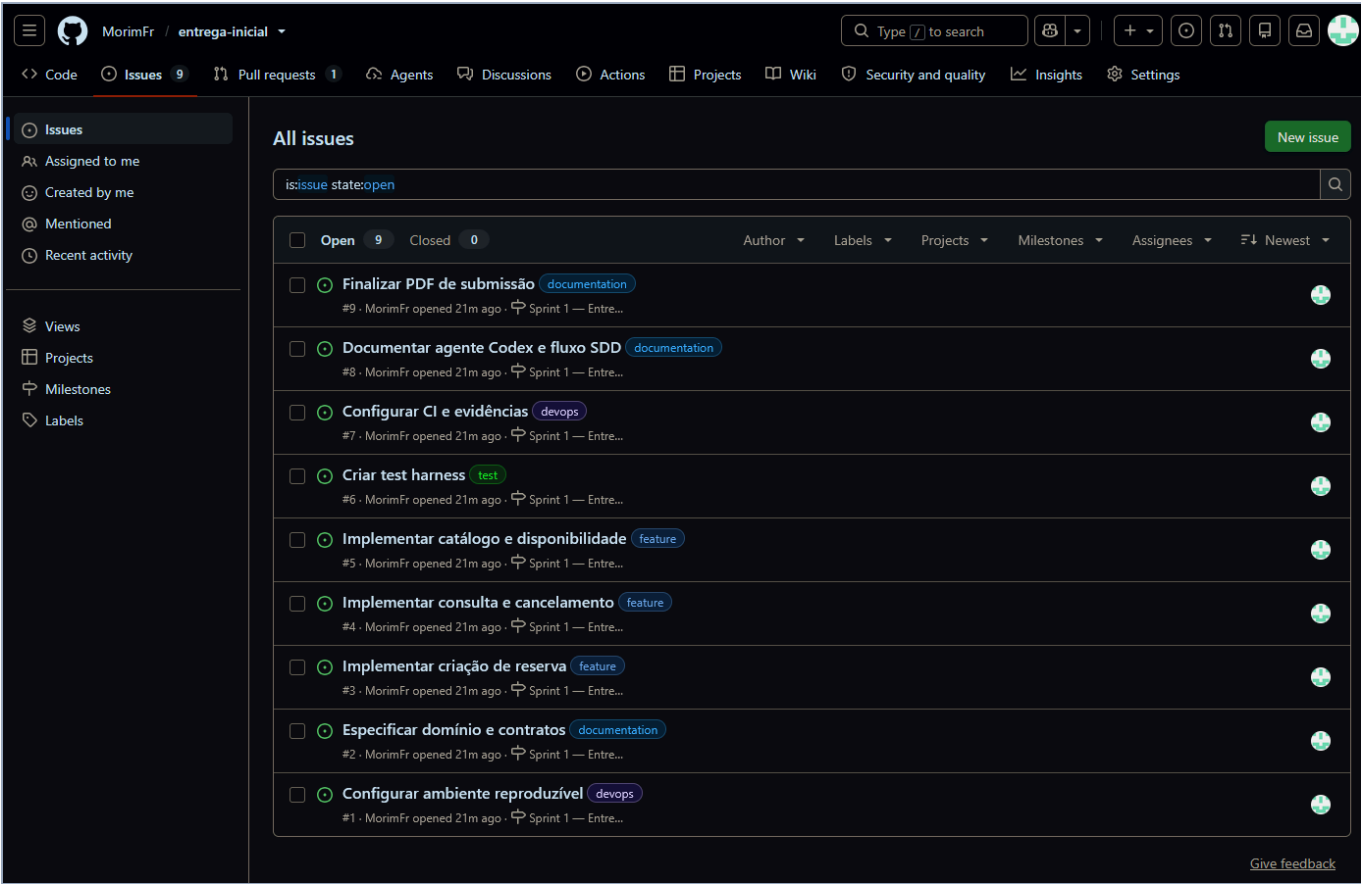


Figura 4 — Nove Issues independentes, categorizadas e vinculadas à milestone Sprint 1.

As Issues decompõem ambiente, especificação, funcionalidades, harness, CI, agente de IA e submissão. Felipe Amorim Monteiro, único integrante do grupo, aparece como responsável. A revisão foi realizada por colaborador externo autorizado, preservando a separação entre autor e aprovador.

8. Evidência — PR da feature para develop

Merged

feat: entrega inicial do CampusFlow #10

SrMorim merged 7 commits into develop from feature/1-entrega-inicial

[data-darkreader-inline-bgcolor] { background-color: var(--darkreader-inline-bgcolor) !important; } [data-darkreader-inline-bgimag

MorimFr commented 25 minutes ago

Owner Author ...

Evidência Docker concluída após o refinamento do Compose: docker compose --profile test run --rm --build tests executou em Python 3.12.14, aprovou 26 testes, atingiu 99,48% de cobertura e retornou exit code 0. Pipeline do commit [c2bfe28](#) também aprovado: <https://github.com/MorimFr/entrega-inicial/actions/runs/33578395082>

SrMorim approved these changes now

View reviewed changes

SrMorim left a comment

Collaborator ...

Aprovado.

SrMorim merged commit d47a101 into develop now

View details Revert

1 check passed

SrMorim deleted the feature/1-entrega-inicial branch now

Restore branch

Pull request successfully merged and closed

You're all set — the branch has been merged.

Add a comment

Write Preview

H B I | | @

Add your comment here...

Markdown is supported Paste, drop, or click to add files

Comment

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

ProTip! Add comments to specific lines under [Files changed](#).

Figura 5 — PR #10 aprovado por SrMorim, CI verde e merge de feature/1-entrega-inicial em develop.

9. Evidência — PR de develop para main

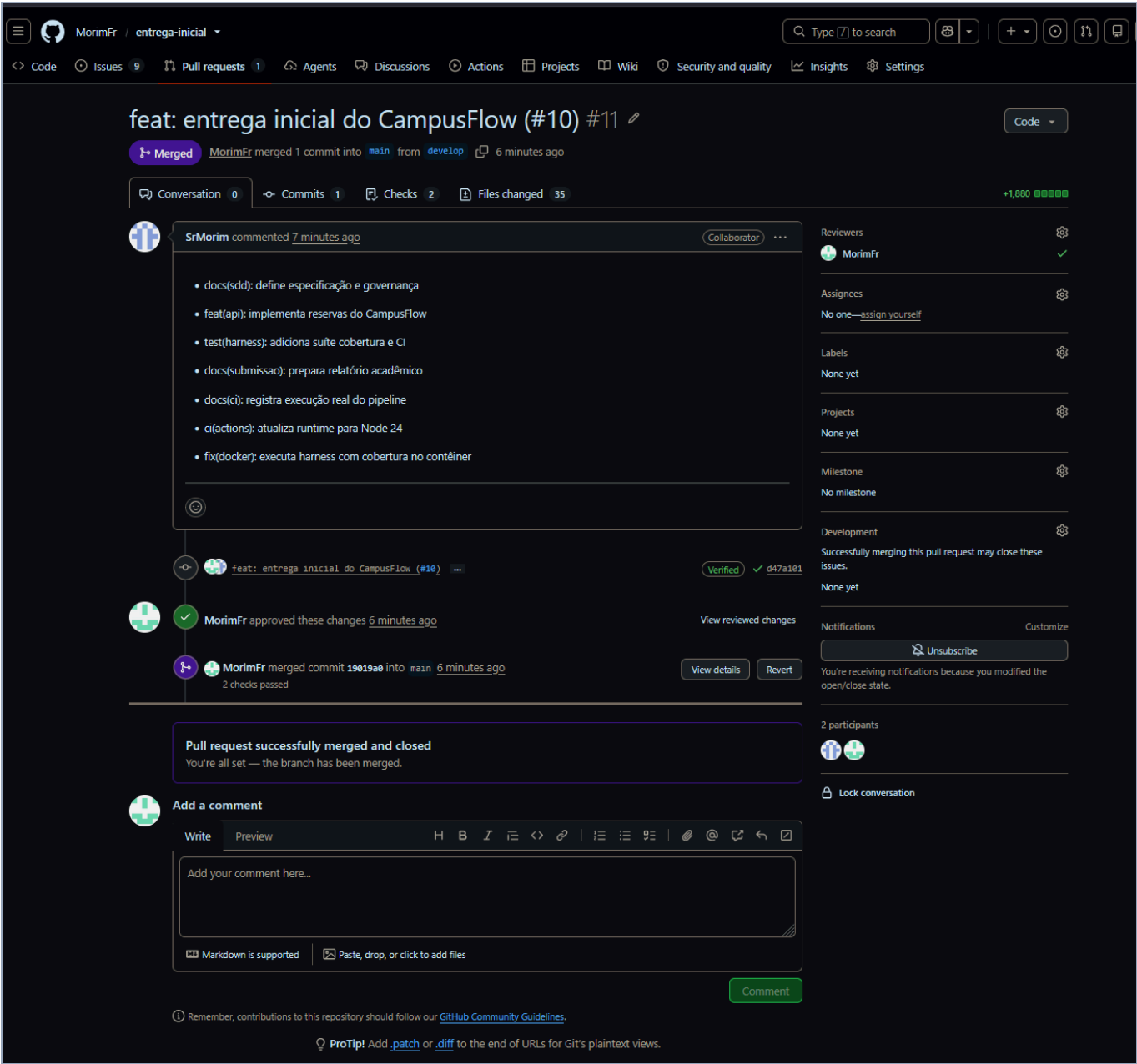


Figura 6 — PR #11 com aprovação, dois checks aprovados e merge de develop em main.

10. Evidência — repositório final

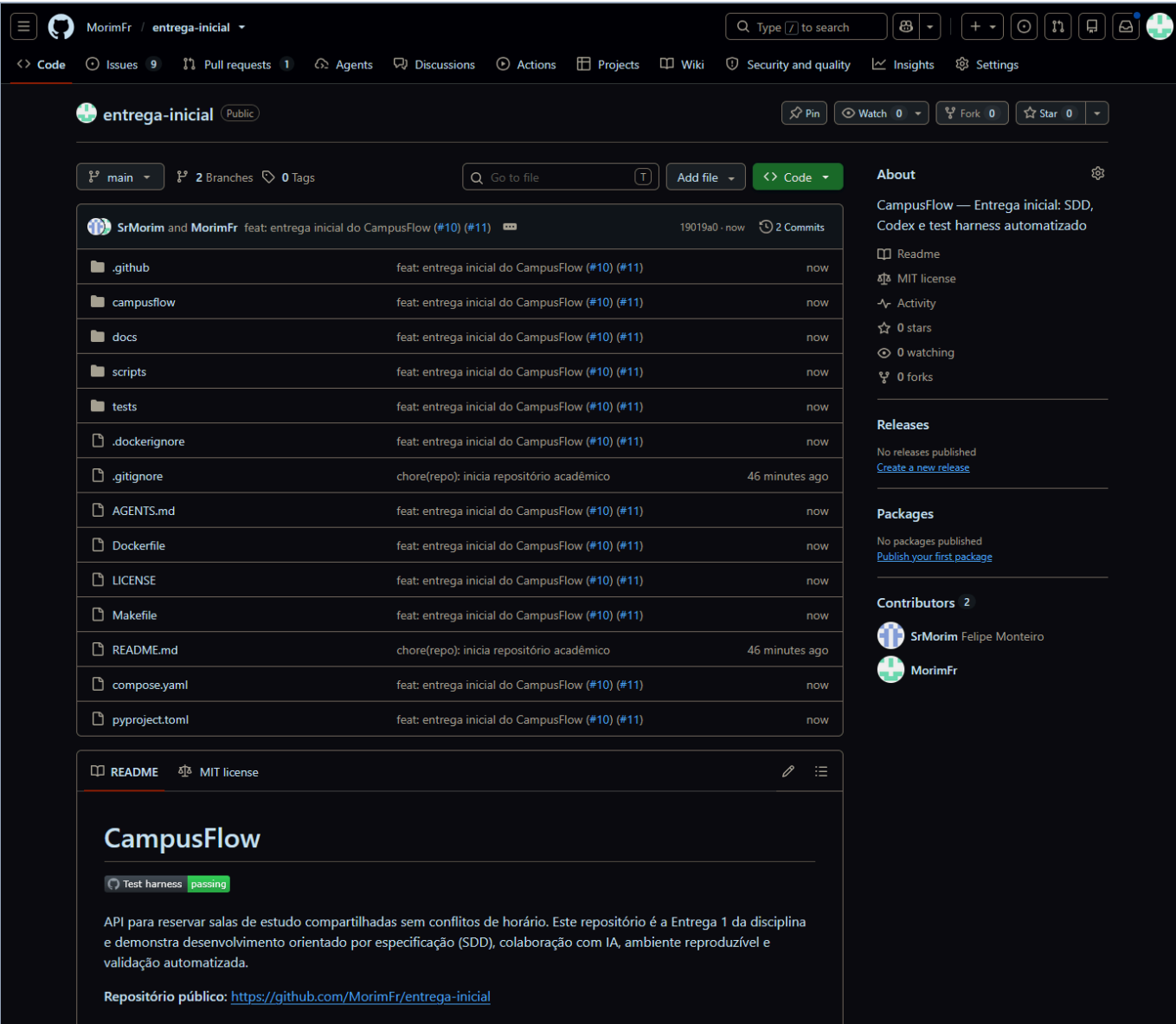


Figura 7 — Branch main após os merges, contendo aplicação, testes, documentação, CI e ambiente.

11. Evidência — proteção da main

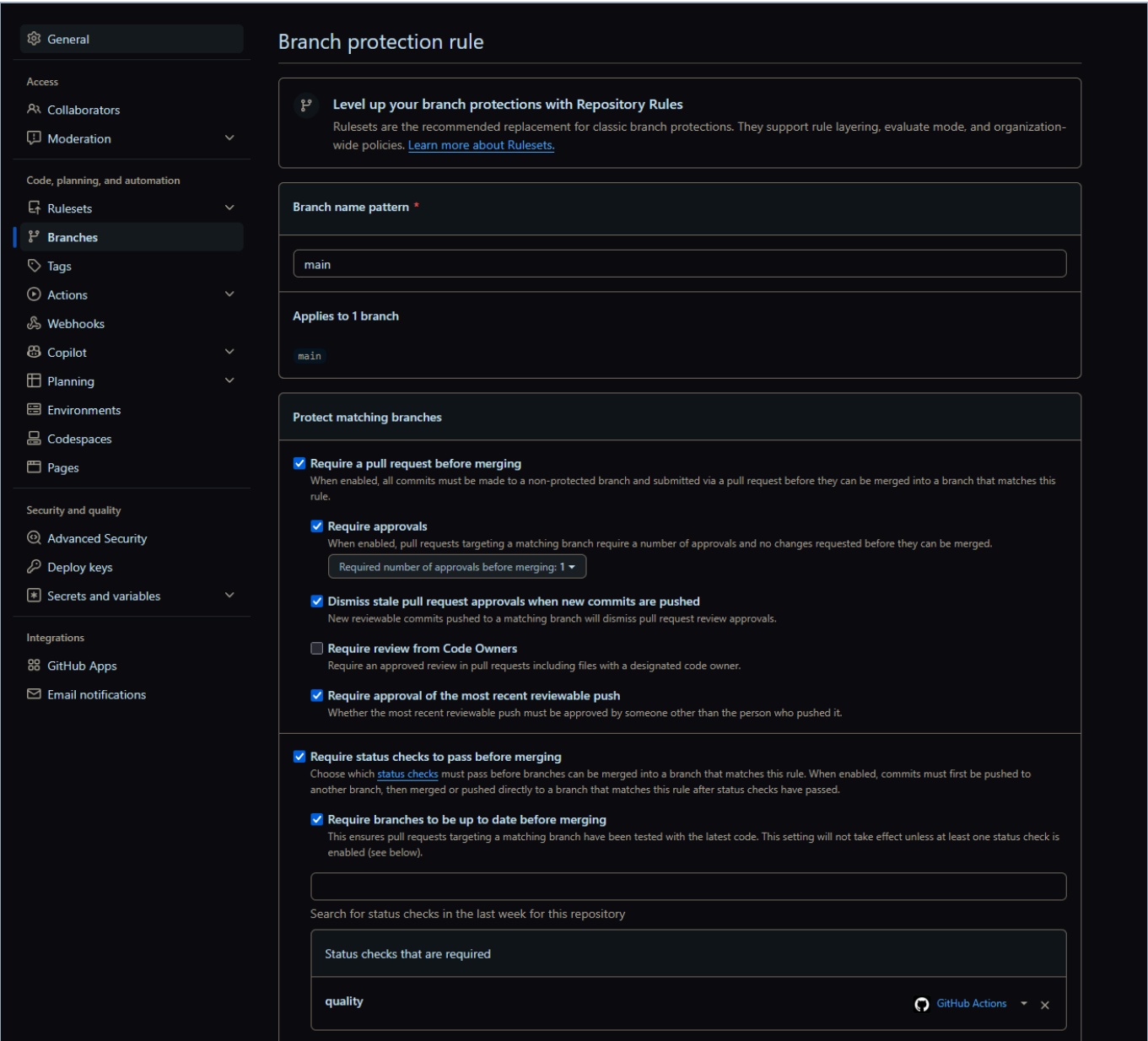


Figura 8 — main exige PR, uma aprovação, atualização da branch e status check quality.

12. Evidência — proteção da develop

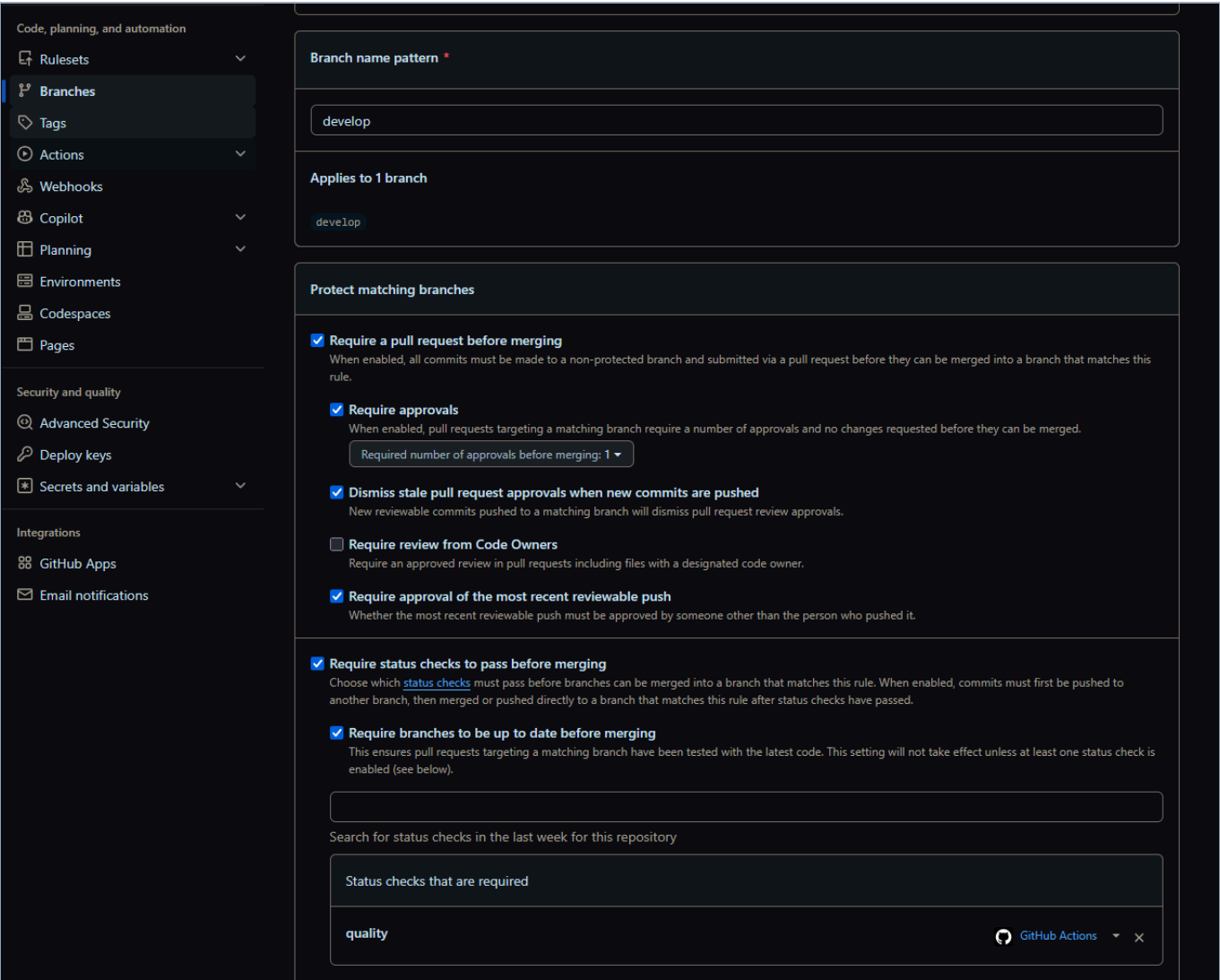


Figura 9 — develop possui os mesmos gates de revisão e validação automatizada.

13. Conclusão

A Entrega 1 estabeleceu uma base funcional e auditável para evolução iterativa do CampusFlow. A especificação dirige o comportamento; a arquitetura separa contratos, regras e persistência; e o harness oferece feedback rápido nos três ambientes de execução.

Atendimento aos itens obrigatórios

Item	Evidência	Estado
Repositório e governança	Issues, branches protegidas, PRs, revisões, README e ADRs.	Atendido
Especificação SDD	Requisitos, regras, contratos, componentes, rastreabilidade e changelog.	Atendido
Agente e ambiente	Codex, AGENTS.md, Dockerfile, Compose e scripts.	Atendido
Test harness	26 testes, bordas, 99,48%, logs, Docker e Actions.	Atendido

Link para avaliação:

<https://github.com/MorimFr/entrega-inicial>

Integrante

Felipe Amorim Monteiro — RA 22452139.